

Databases 1

dr hab. inż. Przemysław Korytkowski, prof. ZUT

Lecture 1 – Relational Database Management System

1

1

Sprawy formalne

- Obecność na wykładach nie jest obowiązkowa...
- Prezentacje w j. angielskim z wykładów są udostępnione w kursie na Moodlu (e-edukacja.zut.edu.pl)
- Kontakt:
 - moje biuro: 108 W11
 - pkorytkowski@zut.edu.pl
 - MS Teams

2

2

Literatura

Podstawowa:

1. Elmasri R., S.B. Navathe, Wprowadzenie do systemów baz danych. Wyd. 7, Helion, Gliwice, 2019.
2. Garcia-Molina H., J.D. Ullman, J. Widom, Systemy baz danych. Kompletny podręcznik. Wyd. 2, Helion, Gliwice, 2011.

Uzupełniająca:

1. Silberschatz A., Korth, HF., Sudarshan, S. (2020) Database System Concepts, 7th ed., McGraw Hill.
2. Beynon-Davies P., Systemy baz danych., WNT, Warszawa, 2003.
3. Petrow A., Database Internals, O'Reilly, 2019.

3

3

Egzamin

- Egzamin z wykładów odbędzie się w formie testu wielokrotnego wyboru.
- **Obowiązują treści przedstawione w literaturze podstawowej.**
- Prezentacje proszę/należy traktować jako materiał niewyczerpujący.

4

4

Course content – Databases 1

1. Relational Database Management Systems
2. Entity-Relationship Diagrams
3. Relational model, Structured Query Language
4. Normalization
5. Storage architectures
6. Indexes
7. Relational algebra, query optimization
8. Transactions and concurrency control
9. Semi-structured data: XML, JSON
10. Database security
11. Data Warehouses

5

5

Course content – Databases 2 (4 semester)

1. Application Programming Interfaces (PHP, Java) for MySQL
2. Object-Relational Mapping
3. NoSQL databases
4. The architecture of distributed databases
5. Document databases – MongoDB
6. Wide column databases – Cassandra
7. Key-value databases – Redis
8. Graph databases – Neo4j

6

6

Course content – Duże zbiory danych (7 semester)

1. Big data, data lake
2. Distributed processing of large data sets across clusters – Apache Hadoop
3. Hadoop Distributed File System
4. Engine for executing data engineering, data science, and machine learning on clusters – Apache Spark
5. Data formats: Parquet, Avro, Delta, Iceberg
6. Distributed event streaming – Apache Kafka
7. Programmable, schedule, and monitor workflows - Apache Airflow

7

7

What is a database?

A database is an organized collection of structured information, or data, usually stored electronically in a computer system, typically describing the activities of a system or systems (companies, organizations).

A database is usually controlled by a **database management system (DBMS)**. Together, the data and the DBMS, along with the applications that are associated with them, are referred to as a database system, often shortened to just database.

A DBMS serves as an interface between the database and its end users or programs, allowing users to retrieve, update, and manage how the information is organized and optimized.

8

8

Drawback of storing data in files

- Usually, we do not have enough of main memory to hold all the data. We must therefore store data in a storage device such as HDD or SSD and bring relevant parts into main memory for processing as needed.
- We have to write special programs to answer each question a user may want to ask about the data.
- We must protect the data from inconsistent changes made by different users accessing the data concurrently.
- We must ensure that data is restored to a consistent state if the system crashes while changes are being made.
- Operating systems provide only a password mechanism for security. This is not sufficiently flexible to enforce security policies in which different users have permission to access different subsets of the data.

9

9

Database vs. spreadsheet

Spreadsheets were originally designed for one user, and their characteristics reflect that. They're great for a single user or small number of users who don't need to do a lot of incredibly complicated data manipulation.

Databases, are designed to hold much larger collections of organized information—massive amounts, sometimes.

Databases allow multiple users at the same time to quickly and securely access and query the data using highly complex logic and language.

10

10

Advantages of databases (DBMSs)

- **Data Independence:** Application programs should not be exposed to details of data representation and storage. The DBMS provides an abstract view of the data that hides such details.
- **Efficient Data Access:** A DBMS utilizes a variety of sophisticated techniques to store and retrieve data efficiently. This feature is especially important if the data is stored on external storage devices.
- **Data Integrity and Security:** If data is always accessed through the DBMS, the DBMS can enforce integrity constraints. For example, before inserting salary information for an employee, the DBMS can check that the department budget is not exceeded. Also, it can enforce access controls that govern what data is visible to different classes of users.

11

11

Advantages of databases (DBMSs)

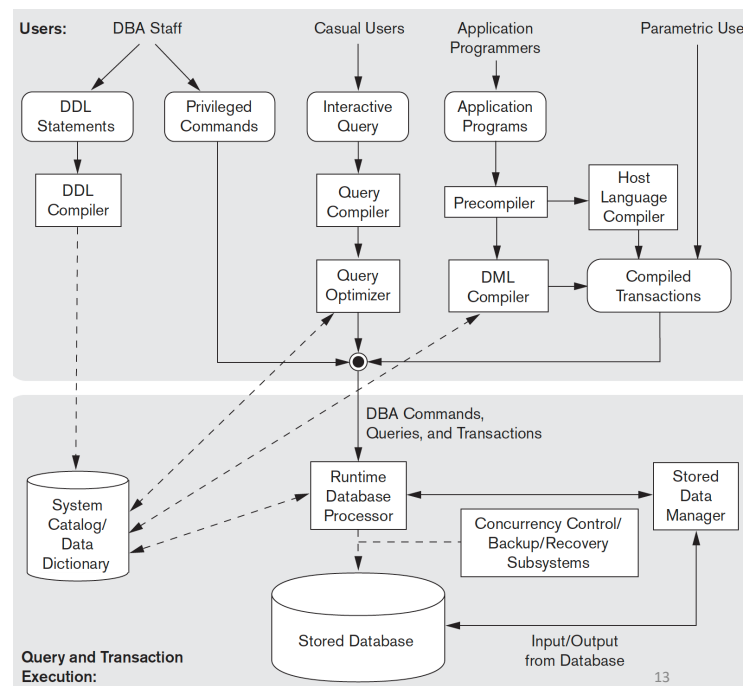
- **Data Administration:** When several users share the data, centralizing the administration of data can offer significant improvements.
- **Concurrent Access and Crash Recovery:** A DBMS schedules concurrent accesses to the data in such a manner that users can think of the data as being accessed by only one user at a time. Further, the DBMS protects users from the effects of system failures.
- **Reduced Application Development Time:** The DBMS offers the high-level interface to the data, which facilitates quick application development. DBMS applications are also likely to be more robust than similar stand-alone applications because many important tasks are handled by the DBMS (and do not have to be debugged and tested in the application).

12

12

Modules of a DBMS

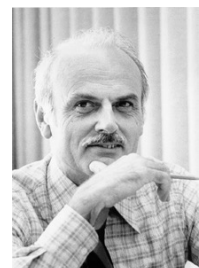
source: Elmasri, R., & Navathe, S. (2014). *Fundamentals of database systems* (Vol. 7). Pearson.



13

Edgar F. (Ted) Codd

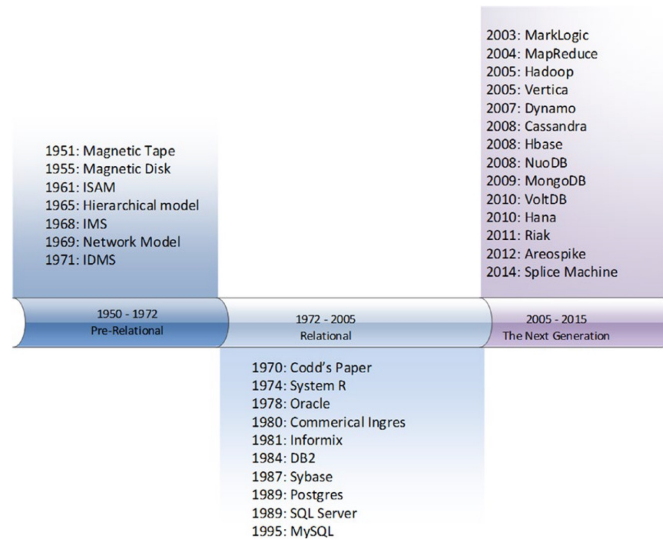
- In 1970, Edgar Codd, at IBM's San Jose Research Laboratory, proposed a new data representation framework called the **relational data model**.
- He proposed a relational DB model with declarative queries and memory.
- Relational = a table with a unique key identifying each row.
- Codd, E.F. A relational model of data for large shared data banks. *Communications of the ACM* 13, 6 (June 1970), 377-387.



source: wikipedia

14

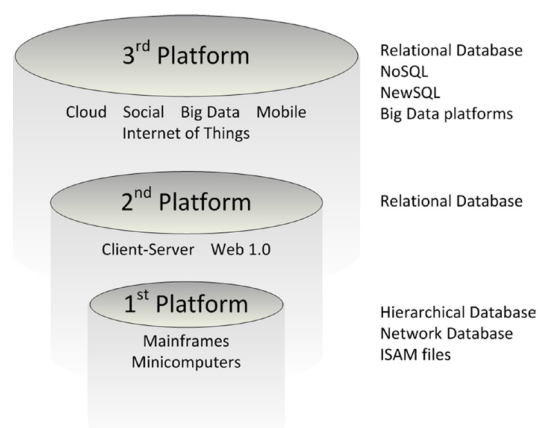
Very short history of databases



Source: Harrison, G. NoSQL, Next Generation Databases, Apress, 2015.¹⁵

15

Three generations of systems



Source: Harrison, G. NoSQL, Next Generation Databases, Apress, 2015.¹⁶

16

DB-Engines Ranking

<https://db-engines.com/en/ranking>

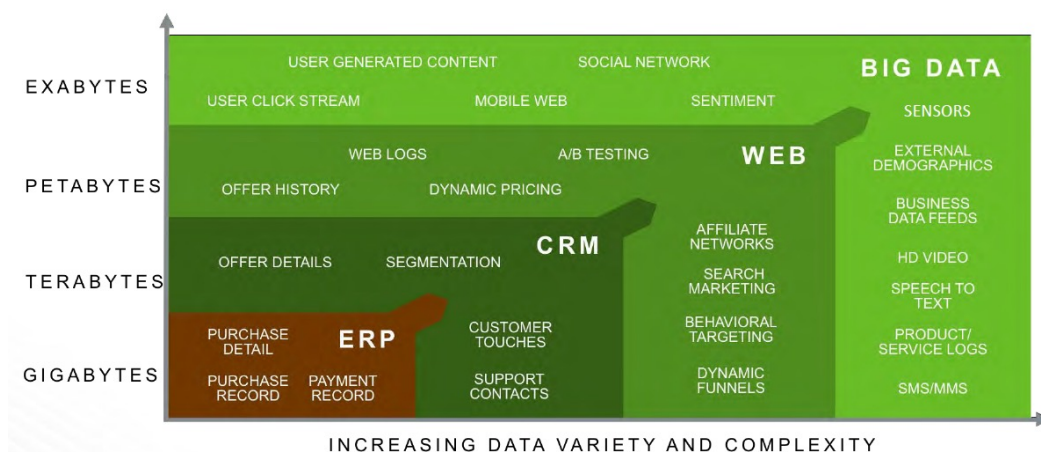
425 systems in ranking, October 2025

Rank			DBMS	Database Model	Score		
Oct 2025	Sep 2025	Oct 2024			Oct 2025	Sep 2025	Oct 2024
1.	1.	1.	Oracle	Relational, Multi-model 	1212.77	+42.15	-96.67
2.	2.	2.	MySQL	Relational, Multi-model 	879.66	-12.11	-143.09
3.	3.	3.	Microsoft SQL Server	Relational, Multi-model 	715.05	-2.27	-87.04
4.	4.	4.	PostgreSQL	Relational, Multi-model 	643.20	-13.97	-8.96
5.	5.	5.	MongoDB 	Document, Multi-model 	368.01	-12.49	-37.20
6.	6.	7.	Snowflake	Relational	198.65	+8.46	+58.05
7.	7.	6.	Redis	Key-value, Multi-model 	142.33	-2.84	-7.30
8.	9.	14.	Databricks	Multi-model 	128.80	+4.74	+43.21
9.	8.	9.	IBM Db2	Relational, Multi-model 	122.37	-1.81	-0.40
10.	10.	8.	Elasticsearch	Multi-model 	116.67	-1.59	-15.18
11.	12.	11.	Apache Cassandra	Wide column, Multi-model 	105.16	-1.82	+7.56
12.	11.	10.	SQLite	Relational	104.56	-3.32	+2.64
13.	13.	15.	MariaDB 	Relational, Multi-model 	87.77	-3.69	+2.88
14.	14.	12.	Microsoft Access	Relational	80.79	-2.82	-11.36
15.	15.	17.	Amazon DynamoDB	Multi-model 	75.91	-4.37	+4.06
16.	16.	16.	Microsoft Azure SQL Database	Relational, Multi-model 	75.49	-3.70	+0.95
17.	17.	18.	Apache Hive	Relational	75.22	-0.88	+22.65
18.	18.	13.	Splunk	Search engine	73.87	-1.89	-17.40
19.	19.	19.	Google BigQuery	Relational	63.20	-2.80	+12.02
20.	20.	21.	Neo4j	Graph	52.51	-1.27	+10.01

17

17

data... big data



Source: www.cloudera.com

18

Companies that thrive on data



19

19

Features of the relational model

- **Entities (records)**, an unordered set of attribute values. In the database system, an entity corresponds to a row and an attribute to a column.
- A **relation** is a collection of different entities corresponding to a table in relational database implementations.
- **Constraints** enforce the consistency of the database. Key constraints are used to identify entities and the relationships between them.
- **Operations on relations**, such as joins, projections, or sums. These operations always return the relations. In practice, this means that a table query returns data in a tabular format.

Relation

	attribute 1	attribute 2	attribute 3
entity 1	data	data	data
entity 2	data	data	data

20

20

Simplified university database

- **Entites:**

- Students
- Courses

- **Relationships:**

- Who takes what?
- How many students attend a course?

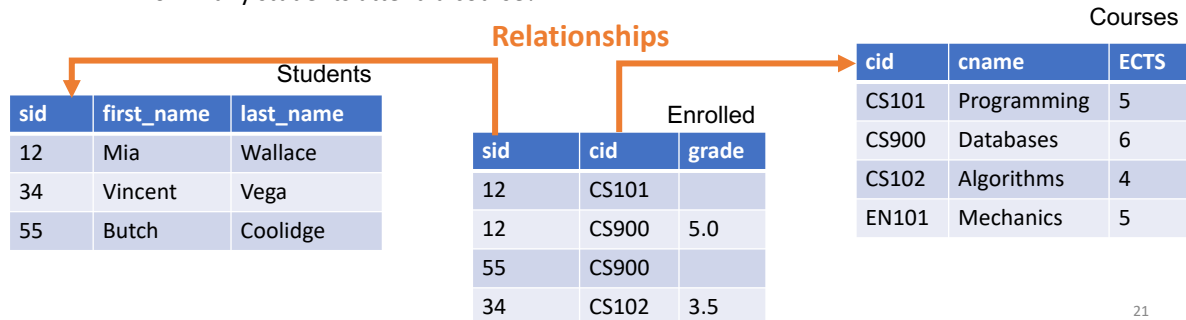
Logical Schema

Students(sid: *int*, first_name: *string*, last_name: *string*)
 Courses(cid: *string*, cname: *string*, ECTS: *int*)
 Enrolled(sid: *int*, cid: *string*, grade: *string*)

relation

attribute

data type



21

21

Three schemas

- **Logical schema** (conceptual)

- Describes the stored data in terms of the data model of the DBMS.
- In a relational DBMS describes all relations that are stored in the database.

- **Physical Schema**

- Summarizes how the relations described in the logical schema are actually stored on secondary storage devices such as disks. We must decide what file organizations to use to store the relations and create auxiliary data structures, called indexes, to speed up data retrieval operations.

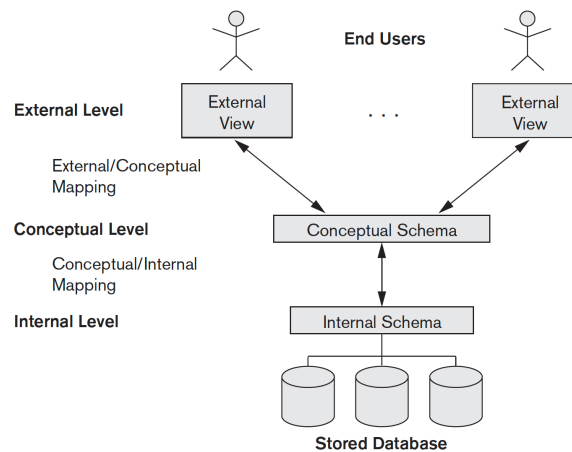
- **External schema**

- Any given database has exactly one logical schema and one physical schema, but it may have several external schemas.
- A view is conceptually a relation, but the records in a view are not stored in the DBMS. Rather, they are computed using a definition for the view, in terms of relations stored in the DBMS.

22

22

Three-schema architecture



source: Elmasri, R., & Navathe, S. (2014). *Fundamentals of database systems* (Vol. 7). Pearson.

23

Data independence

Because separation of logical schema from physical schema applications do not need to worry about *how the data is structured and stored*.

Logical data independence: is the capacity to change the conceptual schema without having to change external schemas or application programs.

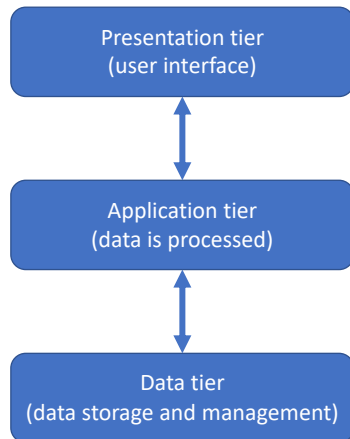
Physical data independence: is the capacity to change the internal schema without having to change the conceptual schema.

One of the most important reasons to use a DBMS

24

24

Logical three-tier client/server architecture



- Is the predominant software architecture for traditional client-server applications.
- The chief benefit is that because each tier runs on its own infrastructure, each tier can be developed simultaneously by a separate development team and can be updated or scaled as needed without impacting the other tiers.

25

25

SQL – Structured Query Language

- SQL is a declarative programming language (4th generation language) used by nearly all relational databases to query, manipulate, and define data, and to provide access control.
- SQL was first developed at IBM by Donald D. Chamberlin and Raymond F. Boyce in the 1970s based on Codd's relational model, which led to implementation of the SQL ANSI standard.
- Based on:
 - relational algebra
 - tuple relational calculus

SQL is popular not only in RDBMS but also in NoSQL and big data.

26

26

Motivation to know SQL

- Dark times about first decade of XXI century.
 - Are relational databases dead?
- Now, as before: everyone sells SQL
 - Cassandra, Apache Spark, Databricks, snowflakes
- “Not-Yet-SQL?”



databricks



snowflake®



27

27

People working with databases

- **Software engineers**
 - Uses DB in other applications
- **Data engineer**
 - Designs data pipelines
- **Systems architects**
 - Designs a software system that stores some data
 - Designs logical schemas
- **Database administrators**
 - Designs logical and physical schemas
 - Is responsible for security and authorization
 - Is responsible for data availability and recovery from failures
 - Is responsible for database tuning
- **Database implementors**
 - Develops new DBMS's, new features

28

28

Thank you for your
attention!

Time for questions

29